Problem No 06

The AMD Zynq UltraScale+ MPSoC is a compelling offering from Xilinx that combines dual or quad-core ARM A53 with dual-core ARM R5F processors with much programmable logic. On the other hand, Intel offers the Agilex family of devices with powerful SoC capabilities, large programmable logic, fast core performances, and low power.

- Intel has rethought the construction of its Agilex FPGAs and SoCs, originally built for performance over power considerations. The initial members of the Agilex series are manufactured with chiplet-based hybrid packaging techniques. A central silicon dies containing the Agilex FPGA's programmable logic fabric, manufactured with Intel's 10nm enhanced SuperFin process, is surrounded with chiplets that add specific I/O features such as fast Ethernet and PCIe SerDes transceivers and CXL. A chiplet stack of high-bandwidth SRAM (HBM) can also be added, as with Agilex M-series devices. Intel's chiplet-based packaging architecture appears below. In addition, Intel Agilex FPGAs and SoCs deliver optimal power, performance, and logic utilization efficiency by integrating hardened protocols for many popular functions, including 100/200/400G Ethernet, PCIe Gen 4/5 interface, Interlaken, CPRI, JESD204B/C, and many more. It has released three versions of agilex in the market. The Agilex family features 2nd Gen Intel Hyperflex™ FPGA architecture for up to 40% higher performance or up to 40% lower power for applications such as data centers, networking, and Edge. Furthermore, Intel Agilex FPGAs and SoCs deliver optimal strength, performance, and logic utilization efficiency by integrating hardened protocols for many popular functions, including 100/200/400G Ethernet, PCIe Gen 4/5 interface, Interlaken, CPRI, JESD204B/C, and many more.



Figure: Intel Agilex FPGA

- However, The AMD Zynq Ultrascale+ MPSoC first MPSoC family is a multi-core chip system. On top of that, it has programmable logic and specific interconnects in the same

chip. So, we can group them into three main elements, the name processing system (PS), programmable logic (PL), and interconnect. We look at the elements one by one.

- **Processing System:** The PS contains processors, fixed peripherals, clocks, memory, and controllers. MPSoC provides quad-core or dual-core variants. The core is ARM Cortex-A53 based. It also features a GPU in the form of ARM Mali-400 MP. There is also a dual-core ARM Cortex-R5 real-time processor in the real-time processing unit. Zynq UltraScale+ MPSoC provides many peripherals, separated into high-speed and low-speed general connectivity.
- **Programmable Logic System: Based on 16nm technology, the PL** has a variant with 103-1143 system logic cells. The Block RAM from 4.5-34.6Mb, Ultra RAM from 0 - 36 Mb, and DSP slices from 240-3528. Some variant has video codec unit. For example, the ultra96 's MPSoC doesn't have a video codec, whereas the costliest ZCU102 has one. The PL includes high-speed connectivity like PCIe Gen3 x16/Gen4 x8 150G Interlaken and 100G Ethernet MAC.

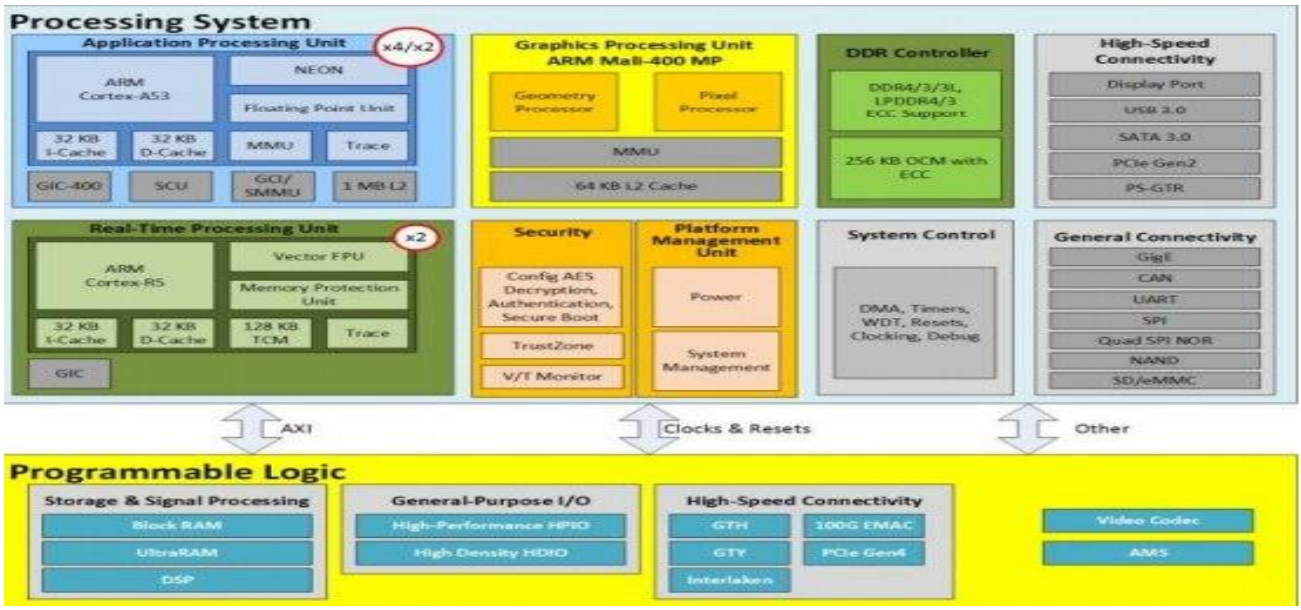


Figure: Main elements of Zynq UltraScale+ MPSoC

So, The AMD ultrascale+ MPSoC, with all these powerful mashups into MPSoC, the family has enabled FPGA to close the gap with ASIC and NVidia GPU, especially for embedded development involving AI, especially current hot topics like convolution neural network (CNN)

The availability of relatively low-price ultra96 makes it practical to try a new design with this board.

⇒ For the automotive /multimedia unit where a number of signal processing and high performance is needed, I will prefer AMD Zynq ultrascale+ MPSoC because of the below reasons. We know today's world is AI-driven, and most signal processing /multimedia unit projects are linked with AI stuffs. AMD zynq ultrascale+ MPSoC .

with all these powerful mashups into MPSoC, the family has enabled FPGA to close the gap with ASIC and NVidia GPU, especially for embedded development involving AI, especially current hot topic like convolution neural network (CNN). The availability of a relatively low-price ultra96 makes it practical to try new designs with this board. Many applications today can be AI-driven, and ZynQ UltraScale+ MPSoC enables us to use FPGA. Full development from scratch is challenging, but prototyping and even hobbyists, including PYNQ support, cut the learning time and implementation time. I would want to believe that the PYNQ performance won't beat the pure manual hand coding, but it does save at least 30% of the time. For a typical project that last one year, 30% is more than one-quarter of savings, and I don't even need to calculate based on the number of engineers needed; multiply by one quarter. AMD zynq ultrascale+ has an application processor with 64 cores, enabling us to do high-performance stuff and signal processing. It is best for scalability while combining real-time control with soft and hard engines for graphics, video, waveform, and packet processing. Built on a common real-time processor and programmable logic-equipped platform, three distinct variants include dual application processor (CG) devices, quad application processor and GPU (EG) devices, and video codec (EV) devices, creating unlimited possibilities for applications such as 5G Wireless, Next-generation ADAS, and Industrial Internet-of-Things. In addition, Zynq UltraScale+ devices become the industry's first adaptive SoCs to be fully certified for use in functional safety applications based on ISO 26262, IEC 61508, and ISO 13849 functional safety standards. The certification of automotive-qualified Zynq UltraScale+ SoCs provides engineers with application-ready devices that meet required functional safety standards.

AMD continues its commitment to automotive functional safety through independent assessments of Zynq UltraScale+ devices. With the previous low-power domain (LPD) ASIL C / SIL3 HFT=1 certification, this device is certified up to ASIL B/SIL2 for the full-power domain (FPD) and up to ASIL B/SC2 certification for programmable logic power domain. So, I prefer AMD zynq ultrascale+ MPsoc for the automotive /multimedia unit with several signal processing and high performance.

Problem No 07

I took problem no two from lab 2 where I integrated a 4-bit ripple carry adder. I designed it with an HDL coder through Simulink in MATLAB. After running it with my Quartus II, I attached the screenshot and code of VHDL.

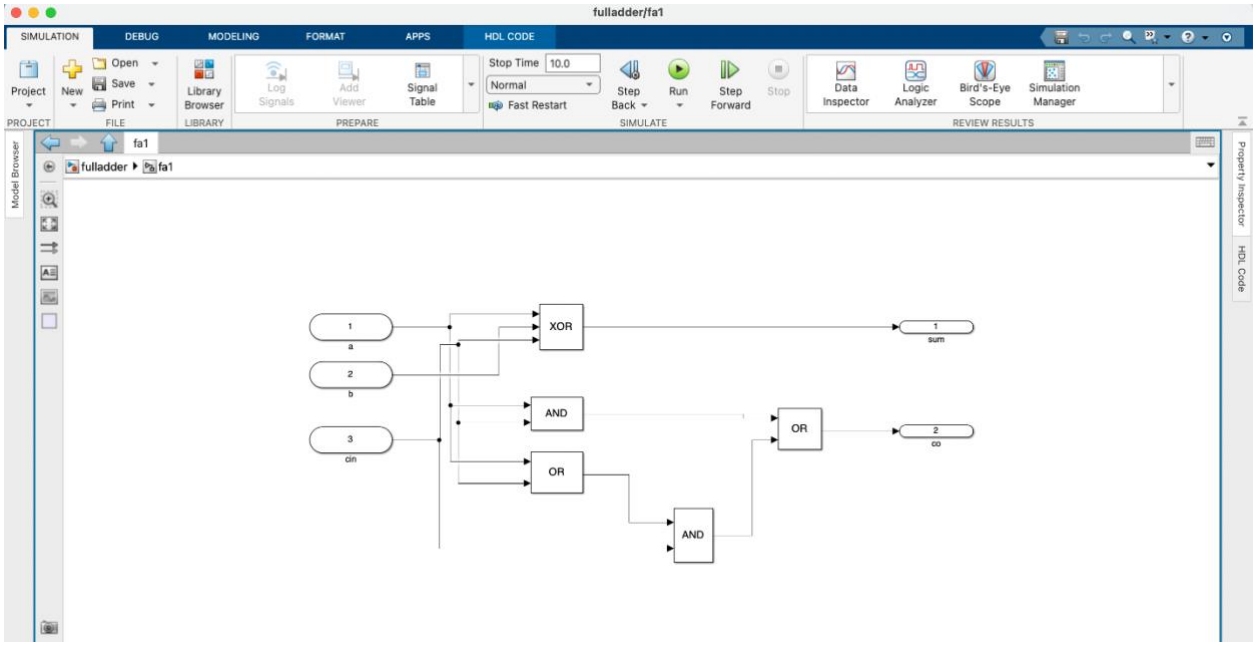


Figure: full adder in MATLAB Simulink

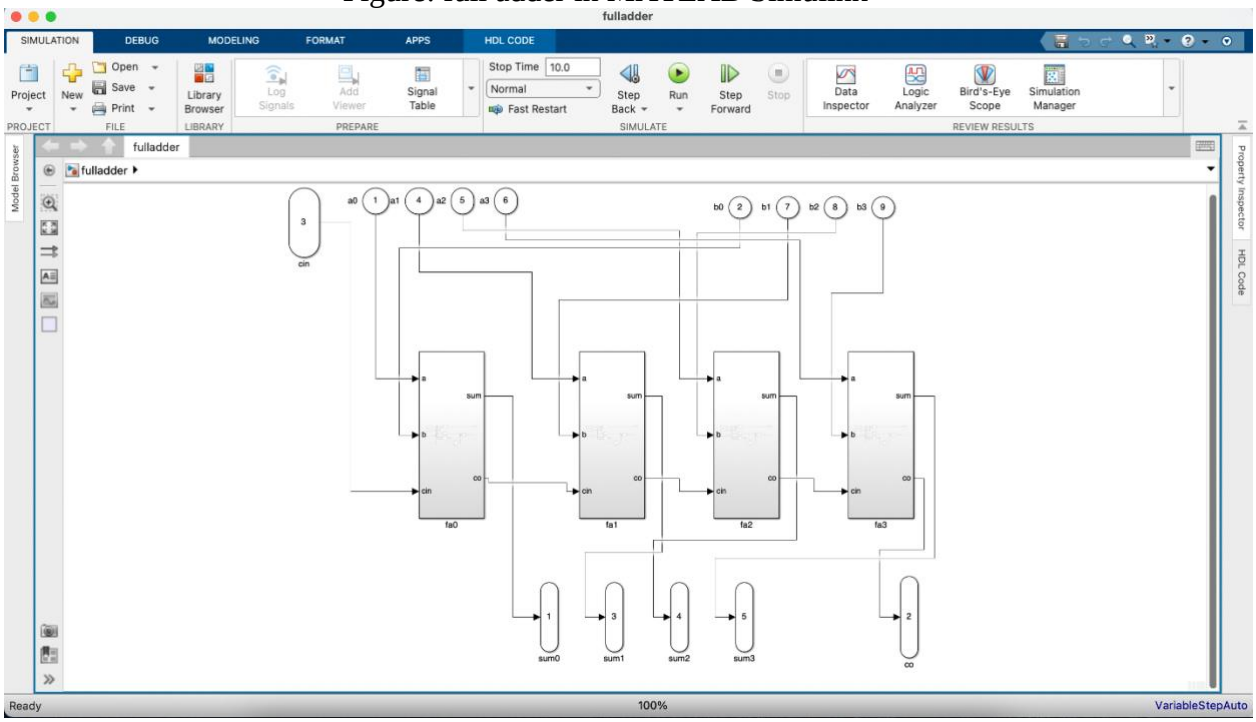
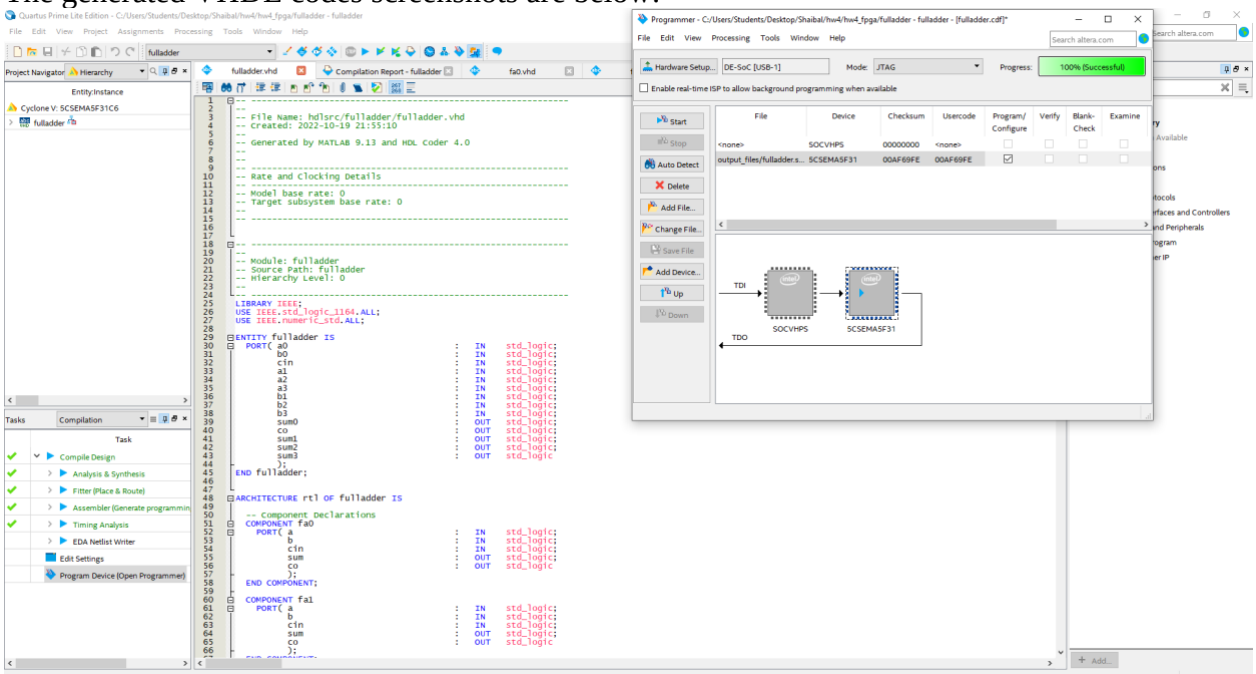


Figure: ripple carry adder in MATLAB Simulink

The generated VHDL codes screenshots are below:



```
58    END COMPONENT;
59
60    COMPONENT fa1
61      PORT( a                         :   IN    std_logic;
62            b                         :   IN    std_logic;
63            cin                       :   IN    std_logic;
64            sum                       :   OUT   std_logic;
65            co                        :   OUT   std_logic
66            );
67    END COMPONENT;
68
69    COMPONENT fa2
70      PORT( a                         :   IN    std_logic;
71            b                         :   IN    std_logic;
72            cin                       :   IN    std_logic;
73            sum                       :   OUT   std_logic;
74            co                        :   OUT   std_logic
75            );
76    END COMPONENT;
77
78    COMPONENT fa3
79      PORT( a                         :   IN    std_logic;
80            b                         :   IN    std_logic;
81            cin                       :   IN    std_logic;
82            sum                       :   OUT   std_logic;
83            co                        :   OUT   std_logic
84            );
85    END COMPONENT;
86
87    -- Component Configuration Statements
88    FOR ALL : fa0
89      USE ENTITY work.fa0(rtl);
90
91    FOR ALL : fa1
92      USE ENTITY work.fa1(rtl);
93
94    FOR ALL : fa2
95      USE ENTITY work.fa2(rtl);
96
97    FOR ALL : fa3
98      USE ENTITY work.fa3(rtl);
99
100   -- Signals
101   SIGNAL fa0_out1                   : std_logic;
102   SIGNAL fa0_out2                   : std_logic;
103   SIGNAL fa1_out1                   : std_logic;
104   SIGNAL fa1_out2                   : std_logic;
105   SIGNAL fa2_out1                   : std_logic;
106   SIGNAL fa2_out2                   : std_logic;
107   SIGNAL fa3_out1                   : std_logic;
108   SIGNAL fa3_out2                   : std_logic;
109
```

```
BEGIN
  u_fa0 : fa0
    PORT MAP( a => a0,
              b => b0,
              cin => cin,
              sum => fa0_out1,
              co => fa0_out2
              );

  u_fa1 : fa1
    PORT MAP( a => a1,
              b => b1,
              cin => fa0_out2,
              sum => fa1_out1,
              co => fa1_out2
              );

  u_fa2 : fa2
    PORT MAP( a => a2,
              b => b2,
              cin => fa1_out2,
              sum => fa2_out1,
              co => fa2_out2
              );

  u_fa3 : fa3
    PORT MAP( a => a3,
              b => b3,
              cin => fa2_out2,
              sum => fa3_out1,
              co => fa3_out2
              );

  sum0 <= fa0_out1;

  co <= fa3_out2;

  sum1 <= fa1_out1;

  sum2 <= fa2_out1;

  sum3 <= fa3_out1;

END rtl;
```

Figure: ripple carry adder main function generated from MATLAB Simulink

The following fa0,fa1,fa2,fa3 full adder blocks functions are below:

fulladder.vhd | Compilation Report - fulladder | fa0.vhd | fa1.vhd | fa2.vhd | fa3.vhd

```
-- -------------------------------------------------------------
--
-- File Name: hdlsrc/fulladder/fa0.vhd
-- Created: 2022-10-19 21:55:10
--
-- Generated by MATLAB 9.13 and HDL Coder 4.0
--
-- -------------------------------------------------------------


-- -------------------------------------------------------------
--
-- Module: fa0
-- Source Path: fulladder/fa0
-- Hierarchy Level: 1
--
-- -------------------------------------------------------------
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;

ENTITY fa0 IS
  PORT( a                                 :   IN    std_logic;
        b                                 :   IN    std_logic;
        cin                               :   IN    std_logic;
        sum                               :   OUT   std_logic;
        co                                :   OUT   std_logic
        );
END fa0;

ARCHITECTURE rtl OF fa0 IS

  -- Signals
  SIGNAL Logical_Operator1_out1           : std_logic;
  SIGNAL Logical_Operator5_out1           : std_logic;
  SIGNAL Logical_Operator3_out1           : std_logic;
  SIGNAL Logical_Operator2_out1           : std_logic;
  SIGNAL Logical_Operator4_out1           : std_logic;

BEGIN
  Logical_Operator1_out1 <= cin XOR (a XOR b);

  Logical_Operator5_out1 <= a AND cin;

  Logical_Operator3_out1 <= a OR cin;

  Logical_Operator2_out1 <= Logical_Operator3_out1 AND cin;

  Logical_Operator4_out1 <= Logical_Operator5_out1 OR Logical_Operator2_out1;

  sum <= Logical_Operator1_out1;

  co <= Logical_Operator4_out1;

END rtl;
```

fulladder.vhd | Compilation Report - fulladder | fa0.vhd | fa1.vhd | fa2.vhd | fa3.vhd

```vhdl
-- -------------------------------------------------------------
--
-- File Name: hdlsrc/fulladder/fa1.vhd
-- Created: 2022-10-19 21:55:10
--
-- Generated by MATLAB 9.13 and HDL Coder 4.0
--
-- -------------------------------------------------------------


-- -------------------------------------------------------------
--
-- Module: fa1
-- Source Path: fulladder/fa1
-- Hierarchy Level: 1
--
-- -------------------------------------------------------------
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;

ENTITY fa1 IS
  PORT( a                                 :   IN    std_logic;
        b                                 :   IN    std_logic;
        cin                               :   IN    std_logic;
        sum                               :   OUT   std_logic;
        co                                :   OUT   std_logic
        );
END fa1;

ARCHITECTURE rtl OF fa1 IS

  -- Signals
  SIGNAL Logical_Operator1_out1           : std_logic;
  SIGNAL Logical_Operator5_out1           : std_logic;
  SIGNAL Logical_Operator3_out1           : std_logic;
  SIGNAL Logical_Operator2_out1           : std_logic;
  SIGNAL Logical_Operator4_out1           : std_logic;

BEGIN
  Logical_Operator1_out1 <= cin XOR (a XOR b);

  Logical_Operator5_out1 <= a AND cin;

  Logical_Operator3_out1 <= a OR cin;

  Logical_Operator2_out1 <= Logical_Operator3_out1 AND cin;

  Logical_Operator4_out1 <= Logical_Operator5_out1 OR Logical_Operator2_out1;

  sum <= Logical_Operator1_out1;

  co <= Logical_Operator4_out1;

END rtl;
```

fulladder.vhd | Compilation Report - fulladder | fa0.vhd | fa1.vhd | fa2.vhd | fa3.vhd

```vhdl
-- -------------------------------------------------------------
--
-- File Name: hdlsrc/fulladder/fa2.vhd
-- Created: 2022-10-19 21:55:10
--
-- Generated by MATLAB 9.13 and HDL Coder 4.0
--
-- -------------------------------------------------------------


-- -------------------------------------------------------------
--
-- Module: fa2
-- Source Path: fulladder/fa2
-- Hierarchy Level: 1
--
-- -------------------------------------------------------------
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;

ENTITY fa2 IS
  PORT( a                                 :   IN    std_logic;
        b                                 :   IN    std_logic;
        cin                               :   IN    std_logic;
        sum                               :   OUT   std_logic;
        co                                :   OUT   std_logic
        );
END fa2;

ARCHITECTURE rtl OF fa2 IS

  -- Signals
  SIGNAL Logical_Operator1_out1           : std_logic;
  SIGNAL Logical_Operator5_out1           : std_logic;
  SIGNAL Logical_Operator3_out1           : std_logic;
  SIGNAL Logical_Operator2_out1           : std_logic;
  SIGNAL Logical_Operator4_out1           : std_logic;

BEGIN
  Logical_Operator1_out1 <= cin XOR (a XOR b);

  Logical_Operator5_out1 <= a AND cin;

  Logical_Operator3_out1 <= a OR cin;

  Logical_Operator2_out1 <= Logical_Operator3_out1 AND cin;

  Logical_Operator4_out1 <= Logical_Operator5_out1 OR Logical_Operator2_out1;

  sum <= Logical_Operator1_out1;

  co <= Logical_Operator4_out1;

END rtl;
```

fulladder.vhd | Compilation Report - fulladder | fa0.vhd | fa1.vhd | fa2.vhd | fa3.vhd

```
-- -------------------------------------------------------------
--
-- File Name: hdlsrc/fulladder/fa3.vhd
-- Created: 2022-10-19 21:55:10
--
-- Generated by MATLAB 9.13 and HDL Coder 4.0
--
-- -------------------------------------------------------------


-- -------------------------------------------------------------
--
-- Module: fa3
-- Source Path: fulladder/fa3
-- Hierarchy Level: 1
--
-- -------------------------------------------------------------
LIBRARY IEEE;
USE IEEE.std_logic_1164.ALL;
USE IEEE.numeric_std.ALL;

ENTITY fa3 IS
  PORT( a                                 :   IN    std_logic;
        b                                 :   IN    std_logic;
        cin                               :   IN    std_logic;
        sum                               :   OUT   std_logic;
        co                                :   OUT   std_logic
        );
END fa3;


ARCHITECTURE rtl OF fa3 IS

  -- Signals
  SIGNAL Logical_Operator1_out1           : std_logic;
  SIGNAL Logical_Operator5_out1           : std_logic;
  SIGNAL Logical_Operator3_out1           : std_logic;
  SIGNAL Logical_Operator2_out1           : std_logic;
  SIGNAL Logical_Operator4_out1           : std_logic;

BEGIN
  Logical_Operator1_out1 <= cin XOR (a XOR b);

  Logical_Operator5_out1 <= a AND cin;

  Logical_Operator3_out1 <= a OR cin;

  Logical_Operator2_out1 <= Logical_Operator3_out1 AND cin;

  Logical_Operator4_out1 <= Logical_Operator5_out1 OR Logical_Operator2_out1;

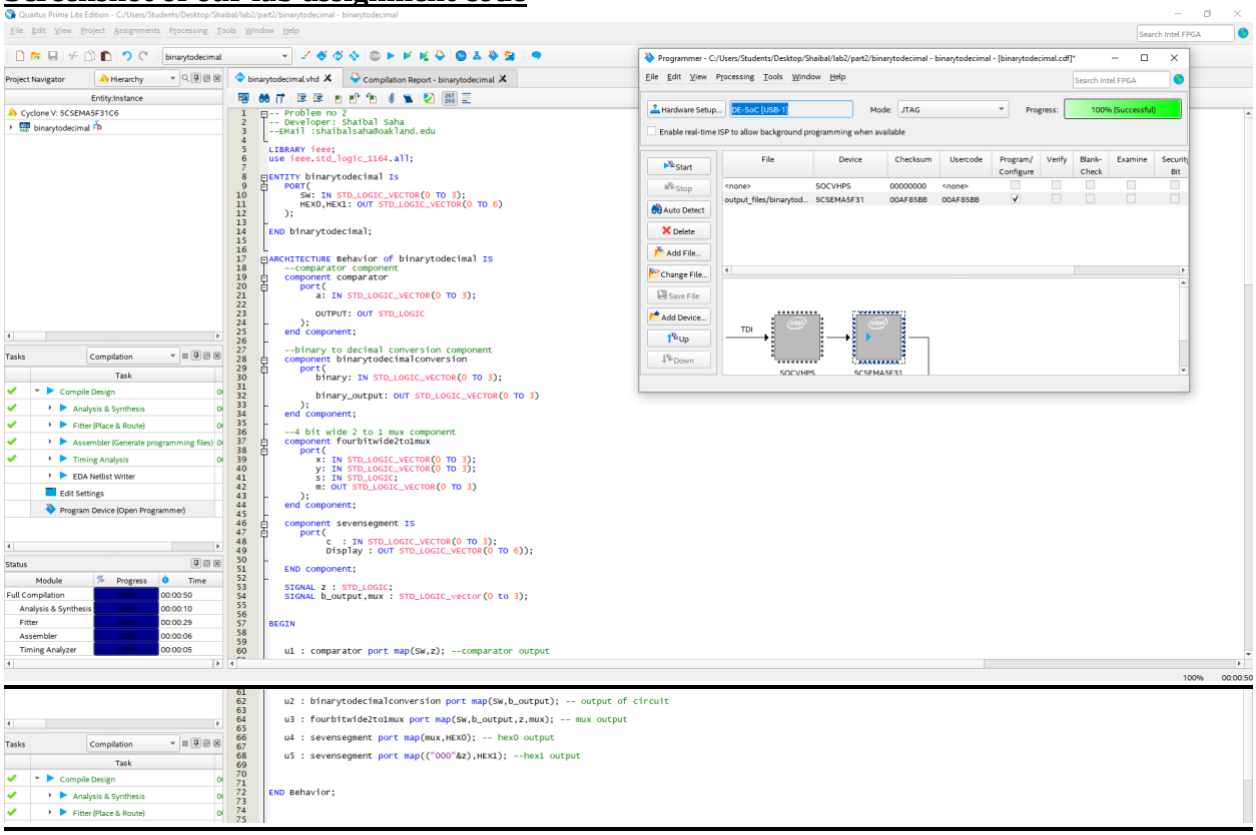
  sum <= Logical_Operator1_out1;

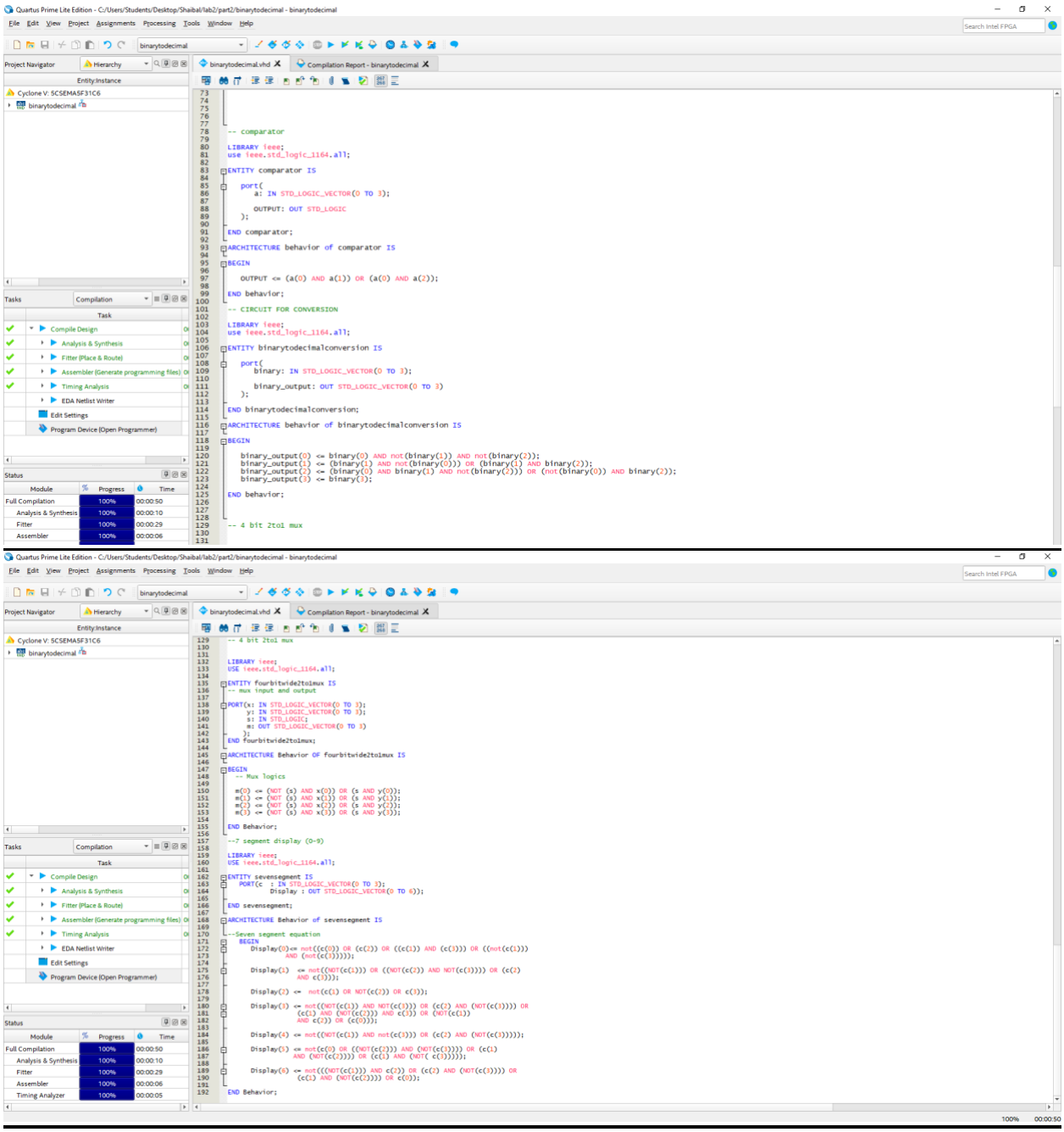
  co <= Logical_Operator4_out1;

END rtl;

```

Screenshot of our lab assignment code





```
-- comparator

LIBRARY ieee;
use ieee.std_logic_1164.all;

ENTITY comparator IS

   port(
      a: IN STD_LOGIC_VECTOR(0 TO 3);

      OUTPUT: OUT STD_LOGIC
   );

END comparator;

ARCHITECTURE behavior of comparator IS

BEGIN

   OUTPUT <= (a(0) AND a(1)) OR (a(0) AND a(2));

END behavior;

-- CIRCUIT FOR CONVERSION

LIBRARY ieee;
use ieee.std_logic_1164.all;

ENTITY binarytodecimalconversion IS

   port(
      binary: IN STD_LOGIC_VECTOR(0 TO 3);

      binary_output: OUT STD_LOGIC_VECTOR(0 TO 3)
   );

END binarytodecimalconversion;

ARCHITECTURE behavior of binarytodecimalconversion IS

BEGIN

   binary_output(0) <= binary(0) AND not(binary(1)) AND not(binary(2));
   binary_output(1) <= (binary(1) AND not(binary(0))) OR (binary(1) AND binary(2));
   binary_output(2) <= (binary(0) AND binary(1) AND not(binary(2))) OR (not(binary(0)) AND binary(2));
   binary_output(3) <= binary(3);

END behavior;


-- 4 bit 2to1 mux
```



```
-- 4 bit 2to1 mux

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY fourbitwide2to1mux IS
-- mux input and output

PORT(x: IN STD_LOGIC_VECTOR(0 TO 3);
     y: IN STD_LOGIC_VECTOR(0 TO 3);
     s: IN STD_LOGIC;
     m: OUT STD_LOGIC_VECTOR(0 TO 3)
   );
END fourbitwide2to1mux;

ARCHITECTURE Behavior OF fourbitwide2to1mux IS

BEGIN
  -- Mux logics

  m(0) <= (NOT (s) AND x(0)) OR (s AND y(0));
  m(1) <= (NOT (s) AND x(1)) OR (s AND y(1));
  m(2) <= (NOT (s) AND x(2)) OR (s AND y(2));
  m(3) <= (NOT (s) AND x(3)) OR (s AND y(3));

END Behavior;

--7 segment display (0-9)

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY sevensegment IS
   PORT(c  : IN STD_LOGIC_VECTOR(0 TO 3);
        Display : OUT STD_LOGIC_VECTOR(0 TO 6));

END sevensegment;

ARCHITECTURE Behavior of sevensegment IS

--Seven segment equation
  BEGIN
     Display(0)<= not((c(0)) OR (c(2)) OR ((c(1)) AND (c(3))) OR ((not(c(1)))
                 AND (not(c(3)))));

     Display(1)  <= not((NOT(c(1))) OR ((NOT(c(2)) AND NOT(c(3)))) OR (c(2)
                 AND c(3)));

     Display(2) <=  not(c(1) OR NOT(c(2)) OR c(3));

     Display(3) <= not((NOT(c(1)) AND NOT(c(3))) OR (c(2) AND (NOT(c(3)))) OR
                          (c(1) AND (NOT(c(2))) AND c(3)) OR (NOT(c(1))
                          AND c(2)) OR (c(0)));

     Display(4) <= not((NOT(c(1)) AND not(c(3))) OR (c(2) AND (NOT(c(3)))));

     Display(5) <= not(c(0) OR ((NOT(c(2))) AND (NOT(c(3)))) OR (c(1)
                 AND (NOT(c(2)))) OR (c(1) AND (NOT( c(3)))));

     Display(6) <= not(((NOT(c(1))) AND c(2)) OR (c(2) AND (NOT(c(3)))) OR
                    (c(1) AND (NOT(c(2)))) OR c(0));

END Behavior;
```

Analysis:

In the lab assignment VHDL and HDL coder, I got HDL coder took a little more time to execute. It is hard to pass the series of inputs in HDL code, whereas it is easy to intel Quartus code. There is limited space to redecorate your automatically generated code in HDL code, but in HDL code, it helps to reduce the number of time to code. In contrast, I assume that if it is a complicated project, Simulink and HDL coder will work much better than the traditional process we work in the lab. HDL coder with Simulink minimizes the line of code. In contrast, I need to spend lots of time fixing the errors in Quartus, and I can give that time to execute the design more significantly and sophistically.

I attached the MATLAB and VHDL files.

Reference

1. https://www.intel.com/content/www/us/en/docs/programmable/683458/current/overview-of-the-fpgas-and-socs.html .
2. https://www.mouser.com/new/intel/intel-agilex/
3. https://community.element14.com/products/manufacturers/xilinx/b/blog/posts/a-first-taste-of-zynq-ultrascale-mpsoc-overview-of-the-family
4. https://www.xilinx.com/products/silicon-devices/soc/zynq-ultrascale-mpsoc.html